

PRÁCTICA FINAL DE LA UNIDAD I

APRENDIZAJE PROFUNDO

Desarrollo de un sistema inteligente de reconocimiento automático de imágenes mediante CNN, Transfer Learning, Fine-Tuning y despliegue web/móvil

Campo	Detalle
Curso	Aprendizaje Profundo
Unidad	I: Fundamentos, redes neuronales artificiales y redes neuronales convolucionales
Semanas	1 a 9
Modalidad sugerida	Trabajo grupal de 3 a 5 estudiantes
Entorno de desarrollo	Kaggle Notebook, Google Colab o Jupyter Notebook
Producto final	Modelo entrenado + aplicación web/móvil + simulación de reconocimiento con datos reales

1. Presentación de la práctica

La presente práctica final integra los conocimientos desarrollados durante la Unidad I del curso de Aprendizaje Profundo. El estudiante deberá diseñar, entrenar, evaluar y desplegar un sistema inteligente capaz de reconocer imágenes reales mediante Redes Neuronales Convolucionales (CNN), Transfer Learning y Fine-Tuning. La práctica culmina con una aplicación web o móvil operativa que permita cargar una imagen, ejecutar inferencia y mostrar la clase predicha con su probabilidad.

2. Objetivo general

Implementar un sistema de reconocimiento automático de imágenes utilizando aprendizaje profundo, redes neuronales convolucionales y modelos preentrenados, evaluando su desempeño mediante métricas de clasificación y desplegando el modelo en una aplicación funcional para realizar simulaciones con datos reales.

3. Resultados de aprendizaje esperados

- Explica las diferencias entre aprendizaje supervisado, no supervisado y por refuerzo en el contexto de la inteligencia artificial.
- Implementa redes neuronales artificiales comprendiendo activación, optimización y retropropagación.
- Diseña una CNN propia para clasificación de imágenes.
- Aplica Transfer Learning y Fine-Tuning usando arquitecturas preentrenadas como EfficientNetB0, MobileNetV2 o ResNet50.
- Evalúa el modelo con accuracy, precision, recall, F1-score, matriz de confusión y tiempo de inferencia.
- Despliega el modelo en una aplicación web o móvil que permita reconocer imágenes reales.

4. Articulación con las semanas de la unidad

Semana	Conocimiento integrado	Evidencia
1	Introducción al aprendizaje profundo; supervisado, no supervisado y refuerzo; arquitecturas ANN.	Cuadro comparativo aplicado al caso seleccionado.
2	Funcionamiento de red neuronal; ReLU, Softmax, Adam; Backpropagation.	Explicación del flujo entrada-proceso-salida.
3	CNN: convolución, kernel, pooling, flatten, dense.	Implementación de CNN propia.
4	Capas convolucionales, stride, padding y pooling.	Diagrama de arquitectura.
5	ResNet, MobileNet, EfficientNet.	Tabla comparativa de modelos.
6	Entrenamiento, validación, curvas de accuracy/loss.	E1: laboratorios prácticos (35%).
7	Clasificación de imágenes, matriz de confusión.	Reporte de resultados.
8	Modelos preentrenados y reconocimiento en tiempo real.	E2: exposición del proyecto CNN (40%).
9	Fundamentos y modelos CNN.	E3: evaluación de la unidad (25%).

5. Caso práctico integrador

Caso recomendado: Sistema inteligente para clasificación de cobertura terrestre mediante imágenes satelitales EuroSAT. El sistema debe reconocer imágenes de uso y cobertura del suelo, tales como bosque, río, área residencial, cultivo anual, vegetación herbácea, carretera, zona industrial, pastizal, cultivo permanente y lago/mar. Esta elección se recomienda por su pertinencia para proyectos de monitoreo ambiental, minería ilegal, agricultura inteligente, turismo territorial y análisis geoespacial aplicado al contexto peruano.

6. Datasets sugeridos y validados

Dataset	Descripción	URL validada	Uso recomendado
EuroSAT	Imágenes Sentinel-2; 10 clases; 27,000 imágenes etiquetadas y georreferenciadas.	https://github.com/phelber/EuroSAT	Clasificación de cobertura terrestre, monitoreo ambiental y teledetección.
PlantVillage	54,306 imágenes de hojas sanas y enfermas para diagnóstico agrícola.	https://github.com/spMohanty/plantvillage-dataset	Detección de enfermedades en plantas y agricultura inteligente.
CIFAR-10	60,000 imágenes de 32x32 píxeles en 10 clases.	https://www.cs.toronto.edu/~kriz/cifar.html	Benchmark educativo para clasificación de objetos.
Intel Image Classification	Imágenes de paisajes naturales organizadas en clases.	https://www.kaggle.com/datasets/puneet6060/intel-image-classification	Clasificación de escenas naturales.
Oxford Flowers 102	Imágenes de 102 categorías de flores.	https://www.robots.ox.ac.uk/~vgg/data/flowers/102/	Clasificación de especies vegetales y reconocimiento visual.

7. Estructura metodológica del proyecto

1. Selección y justificación del dataset.
2. Exploración de datos: clases, cantidad de imágenes, balance y ejemplos visuales.
3. Entrenamiento de una CNN propia desde cero.
4. Entrenamiento con Transfer Learning usando EfficientNetB0.
5. Fine-Tuning descongelando las últimas capas del modelo base.
6. Evaluación integral con métricas y matriz de confusión.
7. Exportación del modelo entrenado.
8. Despliegue web con Streamlit o alternativa móvil.
9. Simulación de reconocimiento usando imágenes reales.

8. Código de entrenamiento en Kaggle o Google Colab

El siguiente código debe ejecutarse por bloques en Kaggle Notebook o Google Colab. Se recomienda activar GPU para obtener mejores tiempos de entrenamiento.

8.1 Configuración del entorno

```
# =====
```

```
# PRÁCTICA FINAL - APRENDIZAJE PROFUNDO
```

```
# Caso: Clasificación de cobertura terrestre con EuroSAT
```

```
# Entorno: Kaggle Notebook / Google Colab
```

```
# =====
```

```
import os
```

```
import zipfile
```

```
import requests
```

```
import numpy as np
```

```

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow.keras import layers, models

from tensorflow.keras.applications import EfficientNetB0, MobileNetV2, ResNet50

from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

from sklearn.metrics import classification_report, confusion_matrix

IMG_SIZE = 224

BATCH_SIZE = 32

EPOCHS = 10

SEED = 123

print("TensorFlow:", tf.__version__)

```

8.2 Descarga y preparación del dataset

```

# =====

# 1. DESCARGA DEL DATASET EUROSAT RGB

# Fuente oficial: https://github.com/phelber/EuroSAT

# =====

url = "https://madm.dfki.de/files/sentinel/EuroSAT.zip"

zip_path = "EuroSAT.zip"

if not os.path.exists(zip_path):

    print("Descargando EuroSAT...")

    response = requests.get(url)

    with open(zip_path, "wb") as f:

        f.write(response.content)

if not os.path.exists("2750"):

    print("Descomprimiendo dataset...")

```

```

with zipfile.ZipFile(zip_path, "r") as zip_ref:

    zip_ref.extractall(".")

```

```
DATA_DIR = "2750"
```

```
print("Dataset listo en:", DATA_DIR)
```

8.3 Carga de datos y clases

```
# =====
```

```
# 2. CARGA DEL DATASET Y PARTICIÓN ENTRENAMIENTO/VALIDACIÓN
```

```
# =====
```

```

train_ds = tf.keras.utils.image_dataset_from_directory(

    DATA_DIR,

    validation_split=0.2,

    subset="training",

    seed=SEED,

    image_size=(IMG_SIZE, IMG_SIZE),

    batch_size=BATCH_SIZE

)

```

```

val_ds = tf.keras.utils.image_dataset_from_directory(

    DATA_DIR,

    validation_split=0.2,

    subset="validation",

    seed=SEED,

    image_size=(IMG_SIZE, IMG_SIZE),

    batch_size=BATCH_SIZE

)

```

```
class_names = train_ds.class_names
```

```
NUM_CLASSES = len(class_names)
```

```

print("Clases:", class_names)

print("Número de clases:", NUM_CLASSES)


AUTOTUNE = tf.data.AUTOTUNE

train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)

val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)

```

8.4 Visualización de imágenes reales del dataset

```

# =====

# 3. VISUALIZACIÓN DE MUESTRAS REALES DEL DATASET

# =====


plt.figure(figsize=(10, 10))

for images, labels in train_ds.take(1):

    for i in range(9):

        ax = plt.subplot(3, 3, i + 1)

        plt.imshow(images[i].numpy().astype("uint8"))

        plt.title(class_names[labels[i]])

        plt.axis("off")

plt.show()

```

8.5 Aumento de datos

```

# =====

# 4. DATA AUGMENTATION

# Mejora la generalización del modelo mediante transformaciones.

# =====


data_augmentation = tf.keras.Sequential([

    layers.RandomFlip("horizontal"),

    layers.RandomRotation(0.1),

    layers.RandomZoom(0.1),

])

```

8.6 Entrenamiento de una CNN propia

```
# =====  
  
# 5. CNN PROPIA DESDE CERO  
  
# =====  
  
cnn_model = models.Sequential([  
    layers.Rescaling(1./255, input_shape=(IMG_SIZE, IMG_SIZE, 3)),  
    data_augmentation,  
  
    layers.Conv2D(32, (3, 3), activation="relu"),  
    layers.MaxPooling2D(),  
  
    layers.Conv2D(64, (3, 3), activation="relu"),  
    layers.MaxPooling2D(),  
  
    layers.Conv2D(128, (3, 3), activation="relu"),  
    layers.MaxPooling2D(),  
  
    layers.Flatten(),  
    layers.Dense(256, activation="relu"),  
    layers.Dropout(0.5),  
    layers.Dense(NUM_CLASSES, activation="softmax")  
)  
  
cnn_model.compile(  
    optimizer="adam",  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"]  
)  
  
cnn_model.summary()
```

```
callbacks_cnn = [
    EarlyStopping(patience=3, restore_best_weights=True),
    ModelCheckpoint("cnn_eurosat.keras", save_best_only=True)
]
```

```
history_cnn = cnn_model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    callbacks=callbacks_cnn
)
```

8.7 Curvas de entrenamiento y validación

```
# =====
```

```
# 6. GRÁFICOS DE ENTRENAMIENTO
```

```
# =====
```

```
def plot_history(history, title):
    plt.figure(figsize=(8, 5))

    plt.plot(history.history["accuracy"], label="Entrenamiento")
    plt.plot(history.history["val_accuracy"], label="Validación")

    plt.title("Accuracy - " + title)
    plt.xlabel("Épocas")
    plt.ylabel("Accuracy")
    plt.legend()
    plt.show()

    plt.figure(figsize=(8, 5))
    plt.plot(history.history["loss"], label="Entrenamiento")
    plt.plot(history.history["val_loss"], label="Validación")
    plt.title("Loss - " + title)
```

```
plt.xlabel("Épocas")

plt.ylabel("Loss")

plt.legend()

plt.show()
```

```
plot_history(history_cnn, "CNN propia")
```

8.8 Transfer Learning con EfficientNetB0

```
# =====
```

```
# 7. TRANSFER LEARNING CON EFFICIENTNETB0
```

```
# =====
```

```
base_model = EfficientNetB0(

    include_top=False,

    weights="imagenet",

    input_shape=(IMG_SIZE, IMG_SIZE, 3)

)

base_model.trainable = False


inputs = layers.Input(shape=(IMG_SIZE, IMG_SIZE, 3))

x = data_augmentation(inputs)

x = base_model(x, training=False)

x = layers.GlobalAveragePooling2D()(x)

x = layers.Dropout(0.3)(x)

outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)

efficient_model = models.Model(inputs, outputs)

efficient_model.compile(

    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),

    loss="sparse_categorical_crossentropy",
```

```

        metrics=["accuracy"]
    )

    callbacks_eff = [
        EarlyStopping(patience=3, restore_best_weights=True),
        ModelCheckpoint("efficientnet_eurosat.keras", save_best_only=True)
    ]

    history_eff = efficient_model.fit(
        train_ds,
        validation_data=val_ds,
        epochs=EPOCHS,
        callbacks=callbacks_eff
    )

    plot_history(history_eff, "EfficientNetB0 - Transfer Learning")

```

8.9 Fine-Tuning y exportación del modelo

```

# =====

# 8. FINE-TUNING

# Descongela las últimas capas del modelo base para ajustar
# características más específicas al dataset EuroSAT.

# =====

base_model.trainable = True

for layer in base_model.layers[:-20]:
    layer.trainable = False

efficient_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.00001),
    loss="sparse_categorical_crossentropy",

```

```

        metrics=["accuracy"]
    )

    history_ft = efficient_model.fit(
        train_ds,
        validation_data=val_ds,
        epochs=5,
        callbacks=callbacks_eff
    )

    plot_history(history_ft, "EfficientNetB0 - Fine-Tuning")

    efficient_model.save("modelo_final_eurosat.keras")
    print("Modelo exportado: modelo_final_eurosat.keras")

```

8.10 Evaluación del modelo

```

# =====

# 9. EVALUACIÓN FINAL: MÉTRICAS Y MATRIZ DE CONFUSIÓN

# =====

loss, accuracy = efficient_model.evaluate(val_ds)
print("Pérdida final:", loss)
print("Accuracy final:", accuracy)

y_true = []
y_pred = []

for images, labels in val_ds:
    predictions = efficient_model.predict(images)
    y_true.extend(labels.numpy())
    y_pred.extend(np.argmax(predictions, axis=1))

```

```

print(classification_report(y_true, y_pred, target_names=class_names))

cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(12, 8))

plt.imshow(cm, interpolation="nearest")

plt.title("Matriz de Confusión - EuroSAT")

plt.colorbar()

plt.xticks(np.arange(len(class_names)), class_names, rotation=45, ha="right")

plt.yticks(np.arange(len(class_names)), class_names)

plt.xlabel("Predicción")

plt.ylabel("Clase real")

plt.tight_layout()

plt.show()

```

8.11 Simulación en notebook con datos reales

```

# =====

# 10. SIMULACIÓN DE RECONOCIMIENTO CON UNA IMAGEN REAL

# La imagen debe estar en el entorno con el nombre imagen_prueba.jpg.

# =====

from tensorflow.keras.preprocessing import image

def reconocer_imagen(ruta_imagen, modelo, clases):

    img = image.load_img(ruta_imagen, target_size=(IMG_SIZE, IMG_SIZE))

    img_array = image.img_to_array(img)

    img_array = np.expand_dims(img_array, axis=0)

    pred = modelo.predict(img_array)

    clase = clases[np.argmax(pred)]

    probabilidad = np.max(pred) * 100

    plt.imshow(img)

```

```

plt.axis("off")

plt.title(f"Predicción: {clase} - {probabilidad:.2f}%")

plt.show()

print("Clase reconocida:", clase)

print("Probabilidad:", round(probabilidad, 2), "%")

# Ejemplo de uso:

# reconocer_imagen("imagen_prueba.jpg", efficient_model, class_names)

```

9. Aplicación web operativa con Streamlit

La aplicación web debe permitir cargar una imagen, visualizarla, ejecutar inferencia y mostrar clase predicha, probabilidad, tiempo de inferencia y modelo utilizado. Guardar el siguiente archivo como app.py.

```

# =====

# app.py

# Aplicación web operativa con Streamlit

# Ejecutar: streamlit run app.py

# =====

import time

import numpy as np

import streamlit as st

import tensorflow as tf

from PIL import Image

from tensorflow.keras.preprocessing import image

st.set_page_config(

    page_title="Reconocimiento Inteligente de Imágenes",

    layout="centered"

)

st.title("Sistema Inteligente de Reconocimiento de Imágenes")

```

```
st.write("Modelo basado en CNN, Transfer Learning, Fine-Tuning y EfficientNetB0.")
```

```
MODEL_PATH = "modelo_final_eurosat.keras"
```

```
IMG_SIZE = 224
```

```
class_names = [  
    "AnnualCrop",  
    "Forest",  
    "HerbaceousVegetation",  
    "Highway",  
    "Industrial",  
    "Pasture",  
    "PermanentCrop",  
    "Residential",  
    "River",  
    "SeaLake"  
]
```

```
@st.cache_resource
```

```
def cargar_modelo():
```

```
    return tf.keras.models.load_model(MODEL_PATH)
```

```
modelo = cargar_modelo()
```

```
archivo = st.file_uploader(  
    "Suba una imagen JPG, JPEG o PNG para realizar el reconocimiento:",  
    type=["jpg", "jpeg", "png"]  
)
```

```
if archivo is not None:
```

```
    img = Image.open(archivo).convert("RGB")
```

```
    st.image(img, caption="Imagen cargada", use_container_width=True)
```

```

img_resized = img.resize((IMG_SIZE, IMG_SIZE))

img_array = image.img_to_array(img_resized)

img_array = np.expand_dims(img_array, axis=0)


inicio = time.time()

prediccion = modelo.predict(img_array)

fin = time.time()


clase = class_names[np.argmax(prediccion)]

probabilidad = np.max(prediccion) * 100

tiempo = fin - inicio


st.subheader("Resultado del reconocimiento")

st.write(f"Clase predicha: **{clase}**")

st.write(f"Probabilidad: **{probabilidad:.2f}%**")

st.write(f"Tiempo de inferencia: **{tiempo:.4f} segundos**")

st.write("Modelo utilizado: **EfficientNetB0 con Fine-Tuning**")


probabilidades = {

    class_names[i]: float(prediccion[0][i])

    for i in range(len(class_names))

}


st.bar_chart(probabilidades)

```

9.1 Archivo requirements.txt

tensorflow

streamlit

pillow

numpy

matplotlib

9.2 Ejecución de la aplicación

En terminal local o servidor:

```
pip install -r requirements.txt
```

```
streamlit run app.py
```

10. Simulación operativa de reconocimiento con datos reales

El estudiante debe realizar como mínimo tres pruebas con imágenes reales externas al dataset de entrenamiento. Las imágenes pueden ser capturadas por el estudiante o descargadas de fuentes abiertas, siempre documentando su procedencia.

Entrada real	Salida esperada	Probabilidad esperada	Interpretación
Imagen real de bosque amazónico o zona forestal	Forest	95% a 99%	El modelo debe reconocer patrones de vegetación densa.
Imagen real de zona urbana de Puno, Juliaca u otra ciudad	Residencial o Industrial	90% a 98%	El resultado puede variar según densidad de construcciones y perspectiva de la imagen.
Imagen real del Lago Titicaca o cuerpo de agua	SeaLake o River	92% a 99%	Debe reconocer predominancia de agua; si hay costa o urbanización puede disminuir la confianza.

11. Entregables obligatorios

- Informe en Word con marco teórico, metodología, capturas, tablas, resultados y referencias APA 7.
- Notebook de Kaggle o Google Colab completamente ejecutable.
- Modelo exportado en formato .keras, .h5 o .pt.
- Aplicación web o móvil operativa.
- Video de 8 a 10 minutos explicando el código, arquitectura, entrenamiento y simulación.
- Presentación final de 10 a 15 diapositivas.

12. Rúbrica de evaluación

Criterio	Peso	Descripción
Marco teórico y revisión de fuentes	15%	Incluye fundamentos de Deep Learning, ANN, CNN, optimización, Transfer Learning y Fine-Tuning con referencias APA 7.
Exploración del dataset	10%	Presenta clases, cantidad de imágenes, balance, visualización y justificación del dataset.
CNN propia	15%	Implementa y explica una CNN desde cero.
Transfer Learning	15%	Implementa modelo preentrenado y compara con CNN propia.

Fine-Tuning	10%	Ajusta capas del modelo base y analiza mejora o pérdida de desempeño.
Evaluación de resultados	10%	Presenta métricas, matriz de confusión y análisis crítico.
Aplicación web/móvil	15%	Permite cargar imágenes reales y ejecutar reconocimiento operativo.
Simulación y defensa	10%	Demuestra inferencia con imágenes reales y explica resultados.

13. Preguntas de análisis obligatorio

- ¿Por qué una CNN es más adecuada que una red neuronal densa tradicional para imágenes?
- ¿Qué función cumplen los filtros o kernels en una convolución?
- ¿Por qué se utiliza ReLU en capas ocultas y Softmax en la capa de salida?
- ¿Qué ventaja ofrece Adam frente al descenso de gradiente clásico?
- ¿Qué diferencia existe entre Transfer Learning y Fine-Tuning?
- ¿Qué evidencia muestra que el modelo generaliza y no solo memoriza?
- ¿Cómo se podría adaptar este sistema a minería ilegal, agricultura inteligente o monitoreo ambiental en Puno?
- ¿Qué limitaciones tendría el sistema si se usa con imágenes muy diferentes al dataset de entrenamiento?

14. Referencias en formato APA 7

Chollet, F. (2021). Deep learning with Python (2nd ed.). Manning Publications.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press. <https://www.deeplearningbook.org/>

Helber, P., Bischke, B., Dengel, A., & Borth, D. (2019). EuroSAT: A novel dataset and deep learning benchmark for land use and land cover classification. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, 12(7), 2217-2226. <https://github.com/phelber/EuroSAT>

Keras. (2026). Keras Applications. <https://keras.io/api/applications/>

Krizhevsky, A. (2009). Learning multiple layers of features from tiny images. University of Toronto. <https://www.cs.toronto.edu/~kriz/cifar.html>

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521(7553), 436-444. <https://doi.org/10.1038/nature14539>

Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using deep learning for image-based plant disease detection. Frontiers in Plant Science, 7, 1419. <https://github.com/spMohanty/plantvillage-dataset>

Streamlit. (2026). st.file_uploader. https://docs.streamlit.io/develop/api-reference/widgets/st.file_uploader

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. Proceedings of the 36th International Conference on Machine Learning. <https://arxiv.org/abs/1905.11946>

TensorFlow. (2026). tf.keras.applications.EfficientNetB0. https://www.tensorflow.org/api_docs/python/tf/keras/applications/EfficientNetB0